

Modélisation et Dynamique d'un Drone Quadrirotor

Pineau Benjamin

5 août 2026

Table des matières

1	Hypothèses	3
2	Repères et Notations	3
2.1	Angles d'Euler	4
3	Modélisation	4
3.1	Forces	4
3.1.1	Poussée des moteurs	4
3.1.2	Moment de rotation des moteurs	5
4	Dynamique	5
4.1	Vitesse angulaire absolue	5
4.2	Distribution des masses	6
4.3	Equations de Newton-Euler	7
4.3.1	Rappel	7
4.4	Résultante	7
4.5	Moments	7
5	PCB	8
5.1	Méthodologie de conception	8
5.1.1	Stack	8
5.1.2	Placement des composants	8
5.1.3	Conception des traces	8
5.2	Table des composants	8
5.2.1	Justification des composants	8
5.2.2	Tables	9
6	Algorithme de contrôle	13
6.0.1	Contrôleur en cascade	13
6.0.2	Design du contrôleur	14
6.0.3	Acceleration Controller	15
6.0.4	Fore and Yaw to Thrust and attitude	15
6.0.5	Poussée Désirée	15
6.0.6	Desired Body Z-axis	15
6.0.7	Orientation Désirée	15
6.0.8	Desired Body Y-axis	15

6.0.9	Desired Body X-axis	16
6.0.10	Desired Rotation Matrix	16
6.0.11	Attitude Controller	16
6.0.12	Rate Controller	16
6.0.13	Efforts vers commandes moteur	17

1 Hypothèses

- Le drone est supposé **rigide**
- Les hélices sont supposées **rigides**
- Le drone possède 2 plans de symétrie : (x, z) et (y, z)
- Le centre des masses G est confondu avec le centre géométrique
- Le moment d'inertie des moteurs et hélices est supposé négligeable devant le reste
- La **Terre** est supposée **plate** et **immobile**.
- La **Terre** est prise comme repère **inertiel**
- L'interaction du drone avec le sol est négligée
- Les capteurs embarqués mesurent par rapport au repère du drone

2 Repères et Notations

On définit deux repères :

- Repère inertiel (x_i, y_i, z_i)
- Repère du drone (x_d, y_d, z_d)

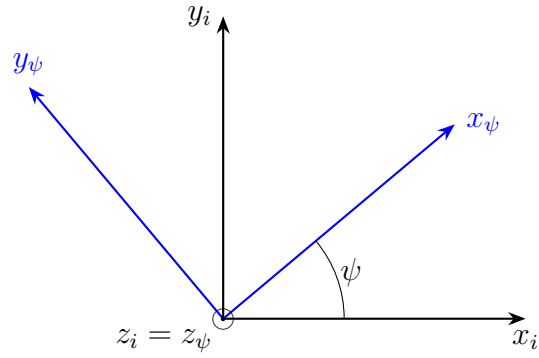
Le système est décrit par 6 paramètres : $q = [x, y, z, \phi, \theta, \psi] = [\xi, \eta]$ avec :

$\xi = [x, y, z]$ les paramètres de position et $\eta = [\phi, \theta, \psi]$ les paramètres d'orientation.

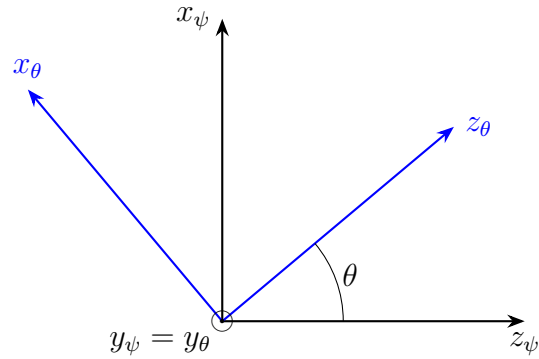
On définit les angles d'Euler suivants :

- ϕ : angle de roulis
- θ : angle de tangage
- ψ : angle de lacet

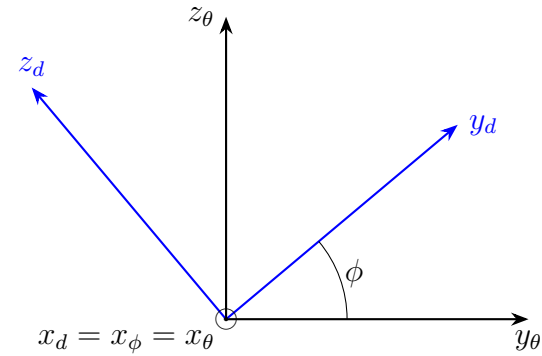
2.1 Angles d'Euler



(a) Rotation selon ψ



(b) Rotation selon θ



(c) Rotation selon ϕ

FIGURE 1 – Suite de rotations (angles d'Euler)

3 Modélisation

3.1 Forces

3.1.1 Poussée des moteurs

La poussée des moteurs est donnée par :

$$T_i = K_T \omega_i^2$$

où K_T est la constante de poussée et ω_i la vitesse angulaire de chaque moteur, et donc :

$$T = T_1 + T_2 + T_3 + T_4 = K_T(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2)$$

3.1.2 Moment de rotation des moteurs

Le moment de rotation des moteurs est donné par :

$$\begin{aligned}\tau_x &= l * k * (\omega_4^2 - \omega_2^2) \\ \tau_y &= l * k * (\omega_3^2 - \omega_1^2) \\ \tau_z &= b * (\omega_2^2 + \omega_4^2 - \omega_1^2 - \omega_3^2)\end{aligned}$$

où l est la distance entre le centre des masses et le moteur, k est la constante de portance.

Et

$$\vec{\tau} = \tau_x \vec{x}_d + \tau_y \vec{y}_d + \tau_z \vec{z}_d$$

4 Dynamique

4.1 Vitesse angulaire absolue

On note $\vec{\Omega} = \dot{\phi} \vec{x}_d + \dot{\theta} \vec{y}_\theta + \dot{\psi} \vec{z}_\psi$ la vitesse angulaire absolue drone par rapport au repère inertiel.

Cependant, cette vitesse utilise des axes intermédiaires. On désire cette vitesse dans le repère du drone (car dans ce repère, l'inertie du drone est constante). Pour ce faire, on définit les matrices de changement de repère (attention avec les matrices de rotation, la convention est inverse) :

$${}^i R_\psi = \begin{pmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

et :

$$\begin{pmatrix} x_\psi \\ y_\psi \\ z_\psi \end{pmatrix} = \begin{pmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix}$$

$${}^\psi R_\theta = \begin{pmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{pmatrix}$$

et :

$$\begin{pmatrix} x_\theta \\ y_\theta \\ z_\theta \end{pmatrix} = \begin{pmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{pmatrix} \begin{pmatrix} x_\psi \\ y_\psi \\ z_\psi \end{pmatrix}$$

$${}^\theta R_\phi = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & \sin \phi \\ 0 & -\sin \phi & \cos \phi \end{pmatrix}$$

et :

$$\begin{pmatrix} x_\phi \\ y_\phi \\ z_\phi \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & \sin \phi \\ 0 & -\sin \phi & \cos \phi \end{pmatrix} \begin{pmatrix} x_\theta \\ y_\theta \\ z_\theta \end{pmatrix}$$

Finalement :

$$c_\phi = \cos \phi, \quad s_\phi = \sin \phi, \quad c_\theta = \cos \theta, \quad s_\theta = \sin \theta, \quad c_\psi = \cos \psi, \quad s_\psi = \sin \psi.$$

$${}^i R_d = \begin{pmatrix} c_\theta c_\psi & c_\theta s_\psi & -s_\theta \\ -c_\phi s_\psi + s_\phi s_\theta c_\psi & c_\phi c_\psi + s_\phi s_\theta s_\psi & s_\phi c_\theta \\ s_\phi s_\psi + c_\phi s_\theta c_\psi & -s_\phi c_\psi + c_\phi s_\theta s_\psi & c_\phi c_\theta \end{pmatrix}$$

On définit le vecteur $\vec{\Omega}_{R_d} = p\vec{x}_d + q\vec{y}_d + r\vec{z}_d$ le vecteur de vitesse angulaire absolue exprimé dans le repère R_d , qui est obtenu en utilisant les matrices de changements de repère :

$$\begin{aligned} \vec{\Omega}_{R_d} &= \dot{\psi} {}^\theta R_\phi {}^\psi R_\theta \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}_{(x_\psi, \vec{y}_\psi, z_\psi)} + \dot{\theta} {}^\theta R_\phi \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}_{(x_\theta, \vec{y}_\theta, z_\theta)} + \dot{\phi} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}_{(x_\phi, \vec{y}_\phi, z_\phi)} \\ \begin{pmatrix} p \\ q \\ r \end{pmatrix} &= \begin{pmatrix} \dot{\phi} - \dot{\psi} S_\theta \\ \dot{\theta} C_\phi + \dot{\psi} C_\theta S_\phi \\ -\dot{\theta} S_\phi + \dot{\psi} C_\phi C_\theta \end{pmatrix} \end{aligned}$$

On extrait la matrice de passage (attention, ce n'est pas une matrice de changement de repère, on a ajouté plusieurs composantes de repères différents, ce n'est plus un objet unique qui se mouvoit) W^{-1} qui lie $\vec{\Omega}_{R_d}$ à $\vec{\Omega}$: $\vec{\Omega}_{R_d} = W^{-1} \vec{\Omega}$

$$W^{-1} = \begin{pmatrix} 1 & 0 & S_\theta \\ 0 & C_\phi & C_\theta S_\phi \\ 0 & -S_\phi & C_\phi C_\theta \end{pmatrix}$$

4.2 Distribution des masses

De part les 2 plans de symétrie du drone, la matrice d'inertie est : $I = \begin{pmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{pmatrix}$

Pour trouver expérimentalement les moments, il faut réaliser l'expérience du *pendule bifilaire* (bifilar pendulum).

4.3 Equations de Newton-Euler

4.3.1 Rappel

La dérivation d'un vecteur $\vec{i}$ dans un repère A par rapport à un repère B est donnée la formule suivante :

$$\frac{{}^B d\vec{i}}{dt} = \frac{{}^A d\vec{i}}{dt} + \vec{\Omega}_A \times \vec{i}$$

où $\vec{\Omega}_A$ est la vitesse angulaire par rapport à B exprimée dans A.

4.4 Résultante

Tout d'abord, posons le vecteur $\vec{v}_d = u\vec{x}_d + v\vec{y}_d + w\vec{z}_d$ qui est la vitesse du drone par rapport au repère inertiel exprimé dans R_d (on exprime la vitesse dans ce repère car les mesures seront faites dans ce repère).

Alors :

$$\begin{aligned} \sum \vec{F} &= m \frac{{}^{R_d} d}{dt} \vec{v}_d = m \left(\frac{{}^{R_d} d}{dt} \vec{v}_d + \vec{\Omega}_{R_d} \times \vec{v}_d \right) \\ \sum \vec{F} &= m \begin{pmatrix} \dot{u} + w(\dot{\theta}C_\phi + \dot{\psi}C_\theta S_\phi) - v(\dot{\psi}C_\theta C_\phi - \dot{\theta}S_\phi) \\ \dot{v} - w(\dot{\phi} - S_\theta \dot{\psi}) + u(-\dot{\theta}S_\phi + \dot{\psi}C_\theta C_\phi) \\ \dot{w} + v(\dot{\phi} - \dot{\psi}S_\theta) - u(\dot{\theta}C_\phi + \dot{\psi}C_\theta S_\phi) \end{pmatrix}_{x_d, \vec{y}_d, z_d} \\ &= m \begin{pmatrix} mgS_\theta \\ -mgC_\theta S_\phi \\ T - mgC_\theta C_\phi \end{pmatrix}_{x_d, \vec{y}_d, z_d} = m \begin{pmatrix} \dot{u} + wq - vr \\ \dot{v} - wp + ur \\ \dot{w} + wp - uq \end{pmatrix}_{x_d, \vec{y}_d, z_d} \end{aligned}$$

Exprimé dans le repère inertiel :

$$\vec{F} = \begin{pmatrix} T(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ T(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ T \cos \phi \cos \theta - mg \end{pmatrix}_{x_i, \vec{y}_i, z_i}$$

4.5 Moments

Pour les moments, on utilise la même méthode et on obtient :

$$\begin{pmatrix} \tau_x \\ \tau_y \\ \tau_z \end{pmatrix}_{x_d, \vec{y}_d, z_d} = \begin{pmatrix} I_x \dot{p} + qr(I_z - I_y) \\ I_y \dot{q} + pr(I_x - I_z) \\ I_z \dot{r} + pq(I_y - I_x) \end{pmatrix}_{x_d, \vec{y}_d, z_d}$$

5 PCB

5.1 Méthodologie de conception

La méthodologie peut être divisée en 2 parties : le placement des composants et la conception des traces.

5.1.1 Stack

La stack utilisée est un 4 couches : une couche pour les signaux (upper), une couche de masse (middle up), une couche de voltage (middle down) et une couche de masse qui peut servir en cas de croisement de traces (bottom).

Le pourquoi : 1) j'uniformise les couches pour ne pas réduire le blindage 2) Signaux en upper pour ne pas se retrouver avec un condensateur qui crée un champ parasite 3) 4 couches car les pcb viennent par 2-4-8 couches, et j'avais besoin au minimum de 3 donc 4.

5.1.2 Placement des composants

Pour ce flight controller, la logique était la suivante : au moment de la conception, le body du drone n'était pas encore fait, donc pour me garder plus d'options j'ai décidé de placer l'IMU au centre de la carte (pour m'assurer qu'il soit au centre géométrique). Ensuite, placer le plus écart le LDO (Low Dropout Regulator, préféré à un hacheur car produit un courant moins bruyé) pour éviter les rayonnements parasites.

Ensuite faire rentrer au mieux les composants dans la carte, en essayant de maximiser l'espace utilisé.

Avec du recul, si je devais refaire cette carte je placerais le STM ou la pièce avec le plus de connexions au centre, et j'agencerais autour. Se retrouver avec le STM sur un côté était bien plus pénible que de juste décaler l'emplacement de la carte.

5.1.3 Conception des traces

Pour les traces, on évite les croisements, aucun angle droit, pour les signaux de faible intensité et basse fréquence : 0.15mm à 0.25mm (en fonction de l'espace disponible) de dégagement, pour les traces de hautes fréquences (clock) ou haute intensité (alimentation) : 0.5mm au maximum disponible, globalement on veut isoler ces pistes le plus possible.

5.2 Table des composants

5.2.1 Justification des composants

- STM32H7 : un microcontrôleur très puissant, pour ne pas être bridé dans le futur car j'ai quelques projets de conduite autonome en tête. Il y a un crystal externe de 8MHz pour les clocks.
- BMP280 : baromètre, capteur de pression qui permet d'estimer l'altitude, utile pour la fusion de données (notamment lors de descente autonome ou alors pour des modèles physiques plus poussés)

- ICM-42688-P : IMU, très performante et peu sensible au bruit, avec le microcontrôleur je trouve que ce sont les 2 pièces maîtresses qui valent le coût d'investir un peu plus dedans
- W25Q128JV : blackbox (mémoire flash), sert à enregistrer les données de vols pour les analyser ensuite. Données accessibles via le port USB-C
- SWD : Serial Wire Debug, optionnel en théorie mais obligatoire en réalité, utile pour flasher ultra simplement le STM, debugger le code, visualiser en temps réel les variables, en bref un must have
- Port UART : il y a un port UART sur la carte pour l'envoi/réception des commandes. Actuellement j'utilise un ESP32 en bluetooth qui communique avec une manette de PS4, car pour le moment je travaille sur un vol contrôlé (pas acro) donc je ne m'attends pas à un éloignement énorme et surtout une antenne est relativement chère pour "juste une antenne" là où un ESP32 sert pour d'autres fonctions.

5.2.2 Tables

Ci-dessous la liste des connexions. Dans certains cas j'aurais effectivement pu privilégier SPI à I2C, mais comme je voulais touchais "à tout" j'ai choisi de diversifier au maximum les protocoles tant que ça n'entravait pas les performances.

BMP280		
Nom du pin	Broche STM32	Fonction
VDD	3V3	Alimentation
GND	GND	Masse
GND	GND	Masse
VDDIO	3V3	Alimentation
SD0	GND	I2C Adresse par défaut
SDI	PB11	I2C2 SDA
CSB	3.3V	VDDIO
SCK	PB10	I2C2 SCL

TABLE 1 – Connexions du BMP280

ICM-42688-P		
Nom du pin	Broche STM32	Fonction
VDD	3V3	Alimentation
VDDIO	3V3	Alimentation
RESV(7)	GND	Masse
RESV(2,3,10)	Not Connected	Pas connecté
AP CS	PA10	GPIO
AP SDI	PB15	SPI2 MOSI
AP SCLK	PA9	SPI2 SCK
AP SDO	PB14	SPI2 MISO
INT1	PB13	EVENTOUT GPIOexti
INT2	GND	Masse

TABLE 2 – Connexions du ICM-42688-P

W25Q128JV		
Nom du pin	Broche STM32	Fonction
VCC	3V3	Alimentation
GND	GND	Masse
CS	PC10	GPIO
WP	3V3	Ouvrir le Write Protection
HOLD or RESET	3V3	Active Low
CLK	PB3	SPI3 SCK
DI	PC12	SPI3 MOSI
DO	PC11	SPI3 MISO

TABLE 3 – Connexions du W25Q128JV

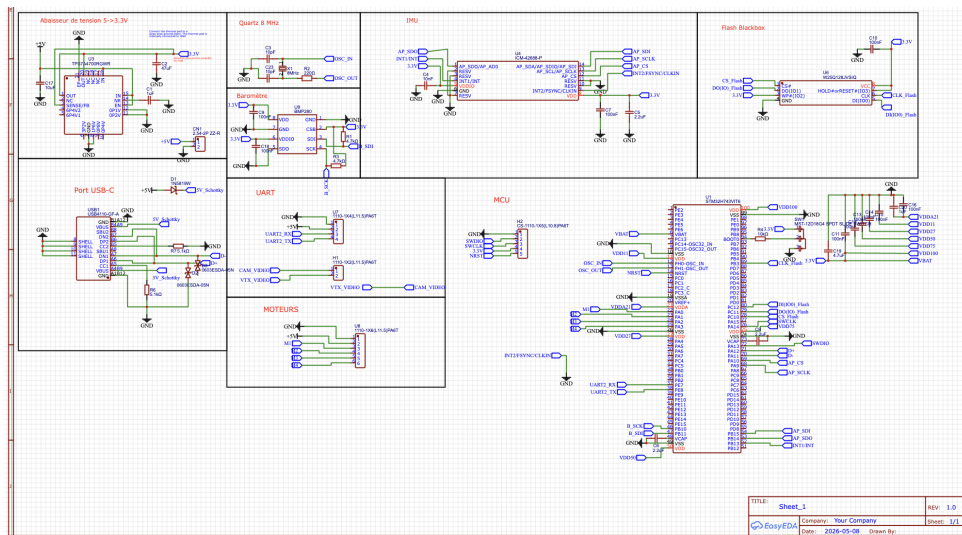


FIGURE 2 – Diagramme du Flight Controller

Signaux Moteurs		
Nom du pin	Broche STM32	Fonction
M1	PA0	TIM5 CH1
M2	PA1	TIM5 CH2
M3	PA2	TIM5 CH3
M4	PA3	TIM5 CH4

TABLE 4 – Connexions du Signaux Moteurs

Signaux de Commande UART		
Nom du pin	Broche STM32	Fonction
Rx	PE7	UART7 Rx
Tx	PE8	UART7 Tx

TABLE 5 – Connexions du Signaux de Commande UART

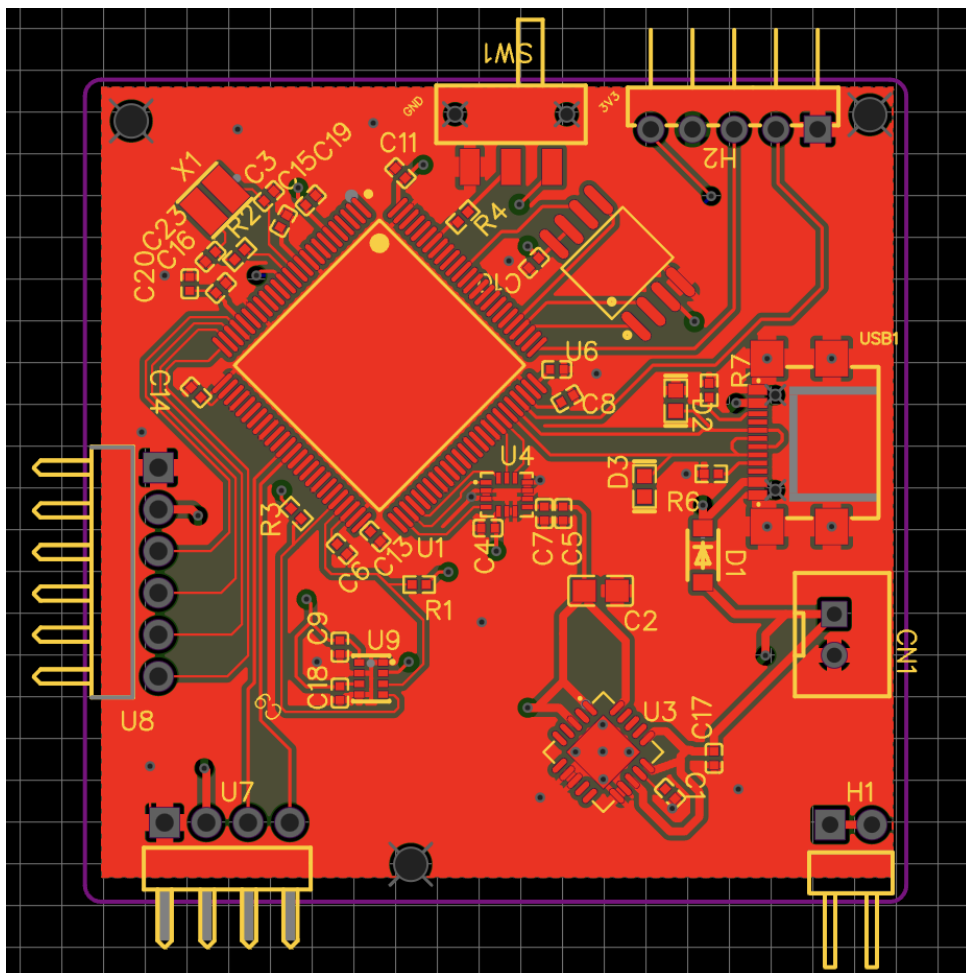


FIGURE 3 – Couche supérieure du PCB (signaux avec composants)

USB-C		
Nom du pin	Broche STM32	Fonction
D+	PA12	OTG FS DP
D-	PA11	OTG FS DM

TABLE 6 – Connexions du USB-C

Serial Wire Debug		
Nom du pin	Broche STM32	Fonction
3V3	3V3	Alimentation
GND	GND	Masse
SWCLK	PA14	Horloge
SWDIO	PA13	Data
NRST	14	Reset

TABLE 7 – Connexions du Serial Wire Debug

6 Algorithme de contrôle

À présent le contrôleur. Partie très intéressante, qui n'a pas une réponse unique donc je vais essayer de détailler le raisonnement. Comme cité précédemment, mon but actuel est de faire un vol contrôlé, càd un vol pas très dynamique mais où les commandes pilotes sont moins directes qu'un vol acro où c'est le pilote qui effectue la majorité des corrections :

le joystick gauche sert à commander l'accélération (ou la poussée) horizontale, le joystick droit contrôle la vitesse angulaire autour de la verticale, et 2 boutons contrôlent l'accélération verticale.

Je vais réécrire les équations de la dynamique pour qu'elles soient plus maniables :

$$m\vec{a} = \vec{F} + m\vec{g}$$

$$m\vec{a} = T^d R_i \vec{z}_d + m\vec{g} \quad (1)$$

$$\vec{\tau} = J\dot{\vec{\Omega}}_{R_d} - \vec{\Omega}_{R_d} \times J\vec{\Omega}_{R_d} \quad (2)$$

(Les effets de l'air autres que la poussée seront traités comme des perturbations et n'apparaissent donc pas dans 1)

Ensuite, introduisons le concept de **contrôleur en cascade** :

6.0.1 Contrôleur en cascade

C'est une technique assez avancée mais très utile donc ça vaut le coup de s'attarder un peu dessus.

Dans un problème usuel d'automatique, on stabilise une unique fonction de transfert, $T(s)$. Cette fonction est généralement issue de la composition de fonction plus simple $H_i(s)$ (de 1er ou 2nd ordre), on peut donc écrire :

$$T(s) = \prod_{i=1}^n H_i(s) \quad (3)$$

Cependant, sous certaines hypothèses, on peut découpler et stabiliser les fonctions indépendamment. On se retrouve donc seulement à stabiliser des fonctions d'ordre 1 ou 2, avec peu de variables d'entrée et de sortie au lieu d'une énorme fonction couplées de partout.

Pour ce faire, il faut que les fonctions opèrent à des vitesses très différentes (il faut un facteur 4, 5 minimum entre les fréquences) et ce dans un ordre hiérarchique.

On part de la boucle la plus rapide (inner loop) puis la deuxième, etc ... jusqu'à la boucle la plus lente (outer loop).

Pourquoi ça fonctionne ? Prenons le cas du drone 4. Les contrôleurs sont des boucles fermées. La boucle la plus lente est le "Acceleration Controller", la plus rapide "Rate Controller".

L'Acceleration Controller produit une attitude désirée. Cette attitude devient la consigne

de l'Attitude Controller, qui cherche à faire converger l'attitude réelle vers cette consigne. L'Attitude Controller génère ensuite une consigne de vitesse angulaire pour le Rate Controller, qui régule les vitesses angulaires du drone.

Maintenant, en supposant que l'inner loop, par rapport à son outer loop, est beaucoup plus rapide, la **consigne est suivie quasiment instantanément du point de vue de l'outer loop**.

Prenons l'exemple du rate controller . Il reçoit en entrée une commande de vitesse angulaire ω_d . Le temps que l'attitude controller effectue sa prochaine itération, le rate controller aura stabilisé le système afin que $\omega = \omega_d$ (Mathématiquement, cela revient à considérer que, du point de vue de la boucle externe, la boucle interne possède un gain apparent proche de 1 : sa sortie suit pratiquement instantanément sa consigne.).

Ainsi, du point de vue de la boucle externe, la boucle interne est quasiment transparente : elle suit sa consigne suffisamment rapidement pour que sa dynamique puisse être négligée lors de la conception du contrôleur externe.

6.0.2 Design du contrôleur

Maintenant, passons au contrôleur. Pour décider de son architecture, je pars des actionneurs et je remonte à la commande. Les actionneurs ici sont les moteurs, donc on peut raccourcir ça à la vitesse de rotation des hélices.

Cela permet de contrôler les couples, ainsi que la poussée du drone. Traitons d'abord les couples. En utilisant 2, on remonte ensuite aux vitesses angulaires du drones. Ces vitesses angulaires s'intègrent pour obtenir l'attitude du drone.

L'attitude du drone, couplée à la poussée du drone, influe sur l'accélération du drone (1), qui est la consigne envoyé par le pilote.

On arrive ainsi à l'architecture globale du contrôleur 4.

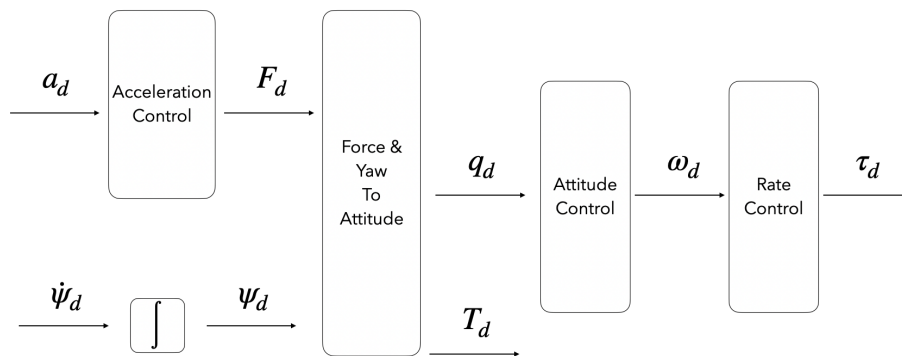


FIGURE 4 – Architecture globale du contrôleur

Maintenant, passons aux design des sous contrôleurs :

6.0.3 Acceleration Controller

En partant de l'équation 1, et en rajoutant un contrôle PI (pour compenser les perturbations) :

$$\vec{F}_d = m \left(\vec{a}_d + K_{p,a} \vec{e}_a + K_{i,a} \int \vec{e}_a dt \right) - m \vec{g} \quad (4)$$

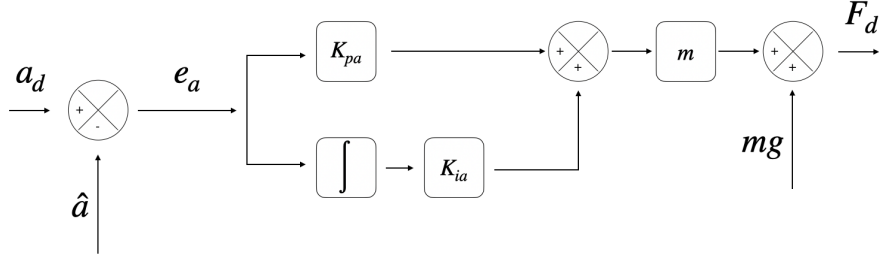


FIGURE 5 – Architecture du contrôleur d'accélération

6.0.4 Fore and Yaw to Thrust and attitude

Ici, ce n'est pas un contrôleur mais seulement des opérations mathématiques pour récupérer l'attitude désirée et la poussée désirée.

6.0.5 Poussée Désirée

$$T_d = \|F_d\|$$

L'attitude désirée se construit en plusieurs étapes :

6.0.6 Desired Body Z-axis

$$b_{3,d} = \frac{F_d}{\|F_d\|}$$

6.0.7 Orientation Désirée

$$b_{1,c} = \begin{bmatrix} \cos \psi_d \\ \sin \psi_d \\ 0 \end{bmatrix}$$

6.0.8 Desired Body Y-axis

$$b_{2,d} = \frac{b_{3,d} \times b_{1,c}}{\|b_{3,d} \times b_{1,c}\|}$$

6.0.9 Desired Body X-axis

$$b_{1,d} = b_{2,d} \times b_{3,d}$$

6.0.10 Desired Rotation Matrix

$$R_d = \begin{bmatrix} b_{1,d} & b_{2,d} & b_{3,d} \end{bmatrix}$$

Cette matrice est ensuite convertie en quaternions q_d .

6.0.11 Attitude Controller

L'erreur de quaternions :

$$q_e = q_d^{-1} \otimes q$$

L'erreur d'attitude (que l'on peut injecter dans un contrôleur contrairement à l'erreur de quaternions)

$$e_R = 2 \operatorname{sgn}(q_{e,w}) q_{e,v}$$

Et donc le contrôleur d'attitude (avec seulement un P car il n'y a pas de perturbations à rejeter)

$$\omega_d = K_R e_R \quad (5)$$

ou bien, si l'on souhaite utiliser le feedforward :

$$\omega_d = K_R e_R + \omega_{ff} \quad (6)$$

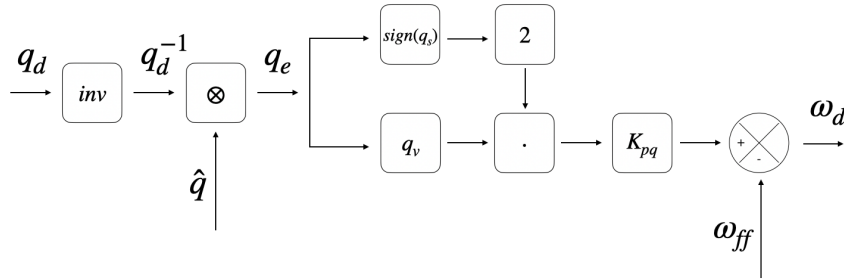


FIGURE 6 – Architecture du contrôleur d'attitude

6.0.12 Rate Controller

L'erreur de vitesse angulaire :

$$e_\omega = \omega_d - \omega$$

Et donc le contrôleur de vitesse angulaire :

$$\tau = K_P e_\omega + K_I \int e_\omega dt + J \dot{\omega}_d \quad (7)$$

Ici aussi on retrouve un terme intégral, qui sert à compenser les perturbations qui peuvent ralentir (ou plus rarement accélérer l'hélice) telles que les effets aérodynamiques, mais aussi la décharge de la batterie qui réduit petit à petit la puissance fournie. Enfin il sert aussi à compenser les incertitudes de modèle.

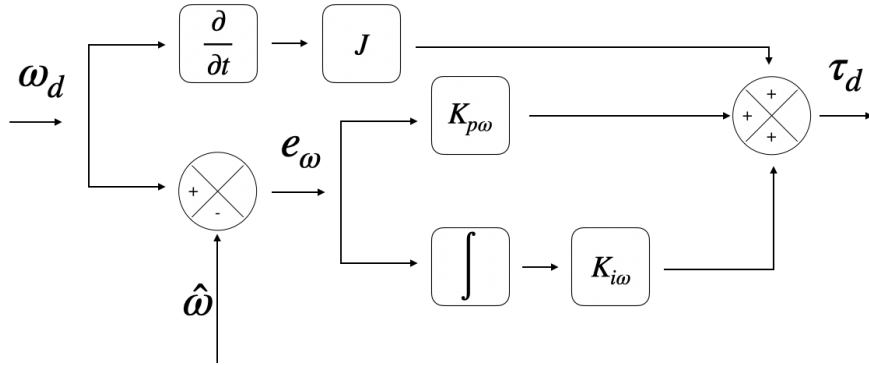


FIGURE 7 – Architecture du contrôleur de vitesse angulaire

6.0.13 Efforts vers commandes moteur

Ensuite, il n'y a plus qu'à résoudre le système :

$$\begin{cases} \tau_x = l * k * (\omega_4^2 - \omega_2^2) \\ \tau_y = l * k * (\omega_3^2 - \omega_1^2) \\ \tau_z = b * (\omega_2^2 + \omega_4^2 - \omega_1^2 - \omega_3^2) \\ Td = K_T(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2) \end{cases}$$

Système de 4 équations de rang 4, il suffit donc d'inverser la matrice des coefficients, on obtient :

$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{2lk} & -\frac{1}{4b} & \frac{1}{4K_T} \\ -\frac{1}{2lk} & 0 & \frac{1}{4b} & \frac{1}{4K_T} \\ 0 & -\frac{1}{2lk} & -\frac{1}{4b} & \frac{1}{4K_T} \\ \frac{1}{2lk} & 0 & \frac{1}{4b} & \frac{1}{4K_T} \end{bmatrix} \begin{bmatrix} \tau_1 \\ \tau_2 \\ \tau_3 \\ T_d \end{bmatrix} \quad (8)$$